

User Acceptance Testing (UAT) Documentation

IITIIM Job Assistant Platform

Milestone 2: LinkedIn Automation & ATS Memory System

Milestone 2.1: LinkedIn Scrapers, Connection Manager & DM Outreach Agent

1. Description & Goal

Automate job discovery and outreach on LinkedIn. The system discovers job posts from LinkedIn's public job board and feed posts, automatically sends connection requests with personalized invitations to job posters, and follows up with AI-personalized direct messages once connection requests are accepted.

2. Codebase Mapping

Role-Based Job Scraper

- File: linkedin-scan.mjs
- Uses Playwright to scan the public LinkedIn jobs board by role/keyword.
- Deduplicates job postings by saving canonical URLs to SQLite jobs.db.
- Executes fully headlessly without requiring a logged-in LinkedIn session, saving API credits.

Feed Post Scraper

- File: linkedin-posts.mjs
- Authenticates on LinkedIn using cookies cached in linkedin-cookies.json.
- Scrapes LinkedIn feed posts containing job announcements.
- Extracts structural details (job title, company, location, poster details, and apply links).
- URL Resolver: Resolves shortened redirect URLs (e.g., lnkd.in, bit.ly) to their final destination targets using HTTP request headers, and cleans tracking variables (utm_*, etc.).
- Text Processing: Normalizes job titles by removing emojis and filtering out common recruiter filler words (like "hiring", "looking for", "join our team").

Connection Manager

- File: linkedin-connect.mjs
- Reads new posts from jobs.db where poster_url is present but connected_at is null.
- Visits the profile pages of the job posters via Playwright.
- Builds a dynamic connection note of under 200 characters containing the poster's first name, role title, and company.
- Finds and clicks the "Connect" button (including checking the "More actions" overflow menu).

- Selects "Other" on the relational prompt, enters the message note, submits the invitation, and records the timestamp in `connected_at`.

DM Follow-Up Agent

- File: `linkedin_followup.py`
- Queries `jobs.db` for profiles where connection invitations were sent (`connected_at` is populated) but no follow-up has been attempted (`followup_sent_at` is null).
- Visits the profiles of those job posters via Playwright.
- Checks if the "Message" button is visible (confirming they accepted the connection request).
- AI Message Generation: Initializes the Anthropic Claude API client. Sends a prompt containing the recruiter's name, company, job title, and a snippet of the original post, as well as candidate information (name, college, title). Claude generates a highly contextual, personalized, plain-text DM (< 300 characters, no emojis or hashtags). If the API key is not configured, it falls back to a high-quality local template.
- Clicks the message compose window, fills the AI-generated message, clicks send, and updates the database record.

3. Database Schema (`jobs.db` — posts & jobs tables)

```
CREATE TABLE IF NOT EXISTS jobs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  company     TEXT NOT NULL DEFAULT '',
  location    TEXT NOT NULL DEFAULT '',
  url         TEXT UNIQUE NOT NULL,
  apply_link  TEXT NOT NULL DEFAULT '',
  source      TEXT NOT NULL DEFAULT 'linkedin',
  keywords    TEXT NOT NULL DEFAULT '',
  scraped_at  TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'new'
);

CREATE TABLE IF NOT EXISTS posts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL DEFAULT '',
  company     TEXT NOT NULL DEFAULT '',
  location    TEXT NOT NULL DEFAULT '',
  apply_link  TEXT NOT NULL DEFAULT '',
  poster_name TEXT NOT NULL DEFAULT '',
  poster_url  TEXT NOT NULL DEFAULT '',
  post_text   TEXT NOT NULL DEFAULT '',
  source      TEXT NOT NULL DEFAULT 'linkedin-posts',
  keywords    TEXT NOT NULL DEFAULT '',
  scraped_at  TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'new',
  post_urn    TEXT UNIQUE DEFAULT NULL,
  post_url    TEXT NOT NULL DEFAULT '',
  apply_url   TEXT NOT NULL DEFAULT '',
  connected_at TEXT DEFAULT NULL,
  followup_sent_at TEXT DEFAULT NULL,
  followup_status TEXT DEFAULT NULL,
  followup_msg TEXT DEFAULT NULL
);
```

4. Verification & Testing Procedure

Job Scraper

1. Run: `npm run linkedin -- --dry-run` to preview jobs without saving.
1. Run: `npm run linkedin -- --role "Software Engineer"` to execute database writes. Verify additions in jobs table.

Feed Post Scraper

1. Run: `npm run posts -- --headed` on the first execution. Complete any LinkedIn 2FA challenge. Verify cookies save to `linkedin-cookies.json`.
1. Inspect posts table to verify resolved redirect URLs, stripped tracking parameters, and cleaned titles.

Connection Manager

1. Run: `npm run connect -- --dry-run` to inspect generated templates under 200 characters.
1. Run: `npm run connect -- --limit 2` to send requests. Confirm page interactions in browser and verify the `connected_at` column is updated.

DM Follow-Up Agent

2. Run: `python linkedin_followup.py --dry-run` to verify API connection and view AI-generated message templates.
2. Run: `python linkedin_followup.py --headed` to verify automated modal navigation, message injection, and database updates.

Milestone 2.2: Job Application Orchestrator & Self-Learning Portal Memory System

1. Description & Goal

Automate job application form filling and submission. The agent detects the ATS portal type, fills form fields based on the candidate's profile, uploads the resume, and clicks submit.

To prevent expensive LLM operations and save API credits, the system implements a Portal Memory System. The first time the agent applies to a portal (Lever, Workday, Custom, etc.), it records the interactive form fields, CSS selectors, and page flow in a structured markdown file (apply_history/<hostname>.md). On subsequent applications to that portal, the file is loaded and injected as context, permitting the agent to apply deterministically.

2. Codebase Mapping

Apply Orchestrator

- File: apply_orchestrator.py
- Selects jobs to apply to. Checks if the candidate is registered in the database as an active paid buyer (database.is_buyer_active()).
- Extracts the hostname from the job's application link.
- Calls portal_memory.get_portal_memory() to find if a cached markdown history file exists.
- Uses ats_detector.detect_from_url() to determine the portal type (Lever, Greenhouse, Workday, Ashby, SmartRecruiters, Google Forms, or Custom).
- Retrieves the corresponding adapter, injects the loaded portal memory, fills in form fields, uploads the resume, submits, and records success logs or redirects failures to the failure queue.
- If no memory file existed, it writes the successfully recorded steps to a new markdown file.

ATS Detector

- File: ats_detector.py
- Uses regular expressions to classify application links by portal pattern (e.g., jobs.lever.co to lever, *myworkdayjobs.com to workday, Google Forms URLs to google_forms).

Portal Memory Manager

- File: portal_memory.py
- Implements normalisation of hostnames (e.g., jobs.eu.lever.co → lever.co).
- Reads and writes markdown files in apply_history/ following the standardized schema (Overview, Flow, Quirks, Human Input, Changelog, and Applied Jobs).

ATS Adapters

- Directory: adapters/

- Files: lever_adapter.py, greenhouse_adapter.py, ashby_adapter.py, smartrecruiters_adapter.py, workday_adapter.py — Deterministic adapters. If a markdown memory file is passed, they extract the selector maps and complete form fields immediately. If not, they explore the fields, fill them, and return recorded_steps in the final ApplyResult.
- File: google_forms_adapter.py — Integrates Google Forms. Rather than exact selectors, it reads fields dynamically, matching labels to the candidate's profile.
- File: custom_adapter.py — General fallback adapter using a browser agent. It reads the <hostname>.md file and injects the text directly into the agent's prompt context, allowing the model to follow the recorded flow instead of using multiple LLM calls to re-explore the site.

Agent Core

- File: agent.py
- Initializes the browser-use Agent framework for automated form filling.

3. Database Schema (jobs.db — applications & failure_queue tables)

```
CREATE TABLE IF NOT EXISTS applications (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  buyer_id   INTEGER NOT NULL,
  post_id     INTEGER,
  job_id      INTEGER,
  portal_hostname TEXT    NOT NULL DEFAULT '',
  ats_type    TEXT    NOT NULL DEFAULT '',
  status      TEXT    NOT NULL DEFAULT 'pending', -- applied | failed |
  paused
  confirmation_id TEXT    DEFAULT '',
  resume_used   TEXT    DEFAULT '',
  cover_letter  TEXT    DEFAULT '',
  applied_at    TEXT,
  notes         TEXT    DEFAULT '',
  created_at    TEXT    NOT NULL
);

CREATE TABLE IF NOT EXISTS failure_queue (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  application_id INTEGER NOT NULL,
  apply_url   TEXT    NOT NULL,
  failure_reason TEXT  NOT NULL,
  failure_type TEXT  NOT NULL,
  buyer_id   INTEGER NOT NULL,
  post_title  TEXT    DEFAULT '',
  company     TEXT    DEFAULT '',
  resolved    INTEGER DEFAULT 0,
  resolved_at TEXT,
  created_at  TEXT    NOT NULL
);
```

4. Verification & Testing Procedure

Unit & Integration Tests

3. Run: `pytest test_portal_memory.py` to verify hostname normalisation, markdown generation, changelog updates, and files parsing.
3. Run: `python test_memory_integration.py` to verify orchestration integration.

Dry Run Orchestrator

2. Run: `python apply_orchestrator.py --dry-run` to verify orchestrator database lookups, paid buyer checks, and ATS detection flows.

Application Learning & Retrieval Loop

2. Select a post ID with a Lever application URL. Verify `apply_history/lever.co.md` is present.
2. Run: `python apply_orchestrator.py --id <post_id> --headed`. Verify in Playwright that the adapter extracts selectors from the markdown file and directly fills the fields without using exploratory loops.
2. Delete `apply_history/lever.co.md`. Run the command again. Verify the adapter explores the form, writes the selectors to `recorded_steps`, submits the form, and generates a fresh `lever.co.md` file.